AFRICAN DEVELOPMENT BANK · DATA INNOVATION FOR AFRICA (DI4A)

Engines of Scale

Big Data Technologies for Statistical Systems

From raw files to trusted indicators: the six building blocks behind every data platform



African Development Bank – Data Innovation for Africa (DI4A)

Why statistical offices need engines

≈ 611 m

Ookla Speedtest tiles

Zoom-16 tiles, fixed and mobile layers, every quarter since 2019

≈ 3 billion

WorldPop 100 m grid

Derived: ≈ 30 million km² × 100 cells per km², per layer and year

Daily

NASA Black Marble radiance

VNP46A2 night-time lights at ~500 m (15 arc-seconds)

The memory wall — and three ways past it

Fits comfortably in RAM

pandas, R — rule of thumb: RAM ≈ 5–10× data size

Fits on one machine's disk

DuckDB, Polars — stream columns from Parquet, out of core

Exceeds one machine, or must be shared

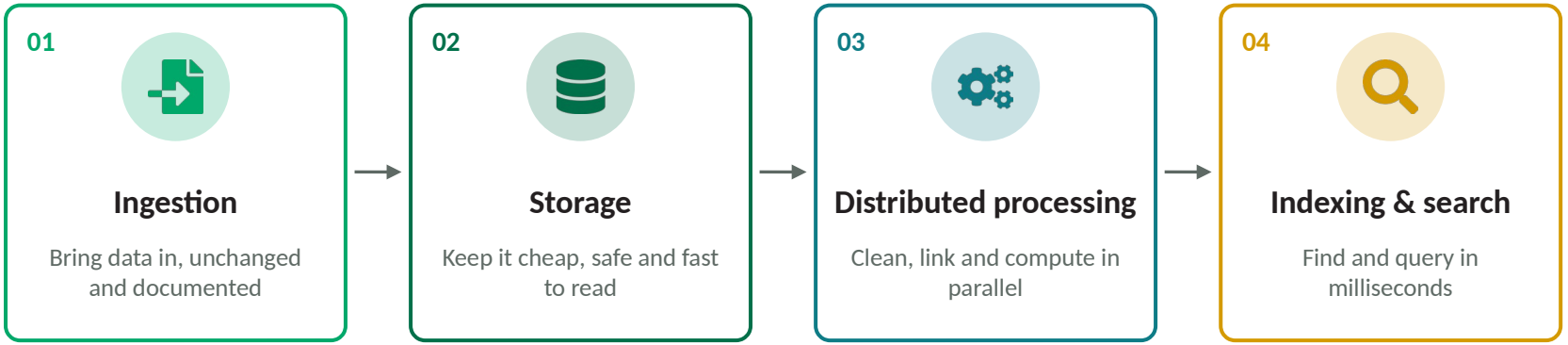
Object storage, Spark, lakehouse

Must answer interactive queries fast

Search engine: Elasticsearch

Sources: Ookla Open Data (GitHub); WorldPop; NASA Black Marble; W. McKinney, "Apache Arrow and the 10 things I hate about pandas" (2017).

Six building blocks of any statistical data platform



05 Orchestration — Schedules every step, manages dependencies, retries failures, raises alerts

06 Metadata & versioning — Describes every dataset, traces every number, versions code, data and environment

Cross-cutting: they span all four blocks above

SECTION 01 OF 06

01

Ingestion

How does data enter the system — and stay trustworthy from the very first byte?



Five ways data enters a statistical system

Batch file transfer

Complete files on a schedule

EXAMPLE

Ookla tiles: quarterly Parquet from a public S3 bucket

Hours–days

API pull

Programmatic requests to a provider

EXAMPLE

Night-time lights from NASA Earthdata or Google Earth Engine

Minutes–days

Change data capture

Only new or changed rows are copied

EXAMPLE

Tax, business and civil registration registers

Minutes

Streaming

Events flow continuously through a log (Kafka)

EXAMPLE

Web price collection, sensors, network events

Seconds

Secure drop zone

Encrypted, access-logged transfer

EXAMPLE

Partner microdata, e.g. telecom operator extracts

Per agreement

Latency ranges are indicative. Ookla bucket: s3://ookla-open-data (AWS Open Data Registry).

The ingestion contract: five rules at the door

1. **Never modify raw data**
   Keep bytes exactly as received; all corrections happen downstream.

2. **Record provenance**
   Source, URL, time and licence (Ookla: CC BY-NC-SA 4.0).

3. **Verify integrity**
   Checksums, row counts, and the expected number of files per period.

4. **Check the schema**
   Ookla added columns in 2022–23: detect drift, never fail silently.

5. **Make loads idempotent**
   Re-running the same quarter gives the same result — never duplicate rows.

ILLUSTRATIVE PROVENANCE RECORD

```
# provenance.json — one per raw file
{
  "source": "Speedtest by Ookla",
  "layer": "mobile",
  "period": "2025-Q4",
  "uri": "s3://ookla-open-data/
    parquet/performance/type=mobile/
    year=2025/quarter=4/",
  "retrieved_at": "2026-01-15T08:12Z",
  "licence": "CC BY-NC-SA 4.0",
  "sha256": "9f2c…e41a",
  "schema_version": "2023-Q3",
  "loaded_by": "pipeline v1.4.0"
}
```

Source: Ookla Open Data documentation (column history and licence).

SECTION 02 OF 06

02

Storage

Where should data live so it is cheap to keep, fast to read and impossible to lose?



Distributed storage: split, replicate, keep work local

Partition

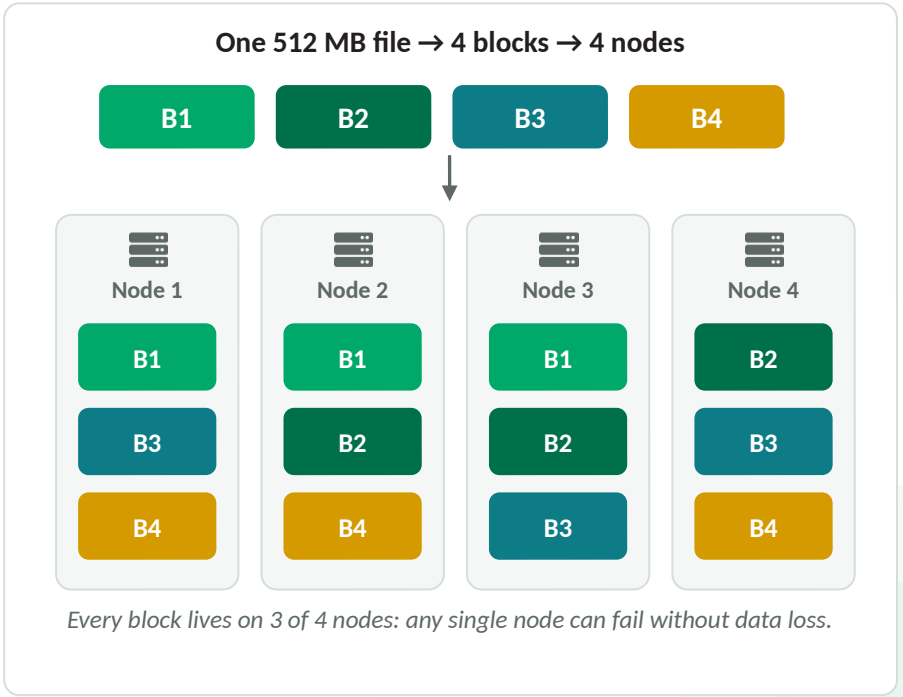
Large files are cut into fixed-size blocks — 128 MB by default in HDFS — spread across nodes.

Replicate

Each block is stored on three nodes by default. Hardware failure is treated as normal.

Keep work local

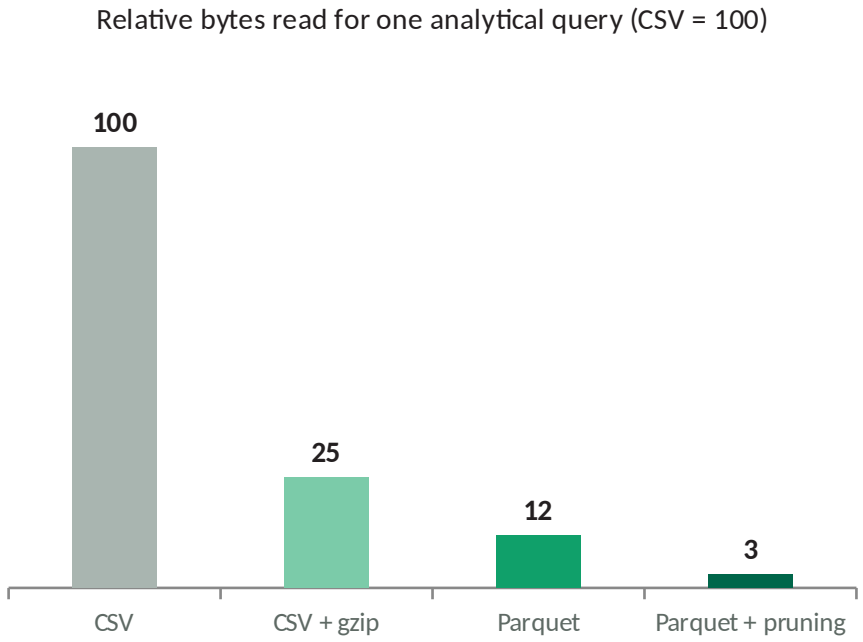
Send the code to the data, not the data to the code — the key to speed at scale.



Every block lives on 3 of 4 nodes: any single node can fail without data loss.

Source: Apache Hadoop HDFS Architecture guide (default block size and replication factor).

File format is a performance decision, not a detail



Read columns, not rows

A tests-weighted mean speed needs 2 of the 11 columns in recent Ookla files.

Compress similar values

Neighbouring values in a column look alike, so columnar files compress far better.

Skip what you don't need

Min/max statistics per row group let the engine skip whole blocks (predicate pushdown).

Illustrative orders of magnitude, not a benchmark. Sources: Apache Parquet documentation; Ookla Open Data schema.

Warehouse, lake, lakehouse

Data warehouse

HOLDS
Curated, structured tables

SCHEMA
Schema enforced on write

STRENGTH
Fast SQL, strong governance

WATCH-OUT
Rigid; costly to scale

EXAMPLES
BigQuery, Snowflake, PostgreSQL

Data lake

HOLDS
Raw files in any format

SCHEMA
Schema interpreted on read

STRENGTH
Cheap, flexible, unlimited

WATCH-OUT
Turns into a "data swamp" without discipline

EXAMPLES
Object storage + Parquet, GeoTIFF, HDF5

Lakehouse

HOLDS
Raw through curated, one place

SCHEMA
On read, plus enforced tables

STRENGTH
Transactions, time travel, schema evolution

WATCH-OUT
Needs engineering maturity

EXAMPLES
Apache Iceberg, Delta Lake, Apache Hudi

Source: Armbrust et al., "Lakehouse: A New Generation of Open Platforms...", CIDR 2021.

The medallion pattern, applied to the lab data

BRONZE

Raw, as received

- Ookla quarterly Parquet files
- WorldPop GeoTIFF rasters
- Black Marble HDF5 granules
- Admin boundary files

SILVER

Validated & linked

- Tiles clipped to the national extent
- Each tile joined to its admin unit
- Population per tile from WorldPop
- Radiance with cloud and quality masks

GOLD

Statistical outputs

- Population-weighted connectivity by region and quarter
- NTL zonal statistics by admin level
- Dashboard and GitHub tables

Quality gate at every arrow: each zone has an owner, a schema contract and tests. Nothing moves forward until they pass.

What a lakehouse table format adds

ACID transactions

Concurrent writers never leave a table half-updated.

Time travel

Query any table as it stood on a past date.

Schema evolution

Add a column without rewriting history.

Partition pruning

A query for one quarter skips all the others.

SPARK SQL + APACHE ICEBERG

```
-- The table exactly as it stood
-- when the Q2 figures were published
SELECT admin1, dl_kbps_weighted
FROM   lake.gold.connectivity
TIMESTAMP AS OF '2026-07-15 00:00:00'
WHERE  year = 2026 AND quarter = 2;
```

Why statisticians care

A revision can be explained by re-running the exact input state of a published number.

Sources: Apache Iceberg and Delta Lake documentation. Table and column names are illustrative.

SECTION 03 OF 06

03

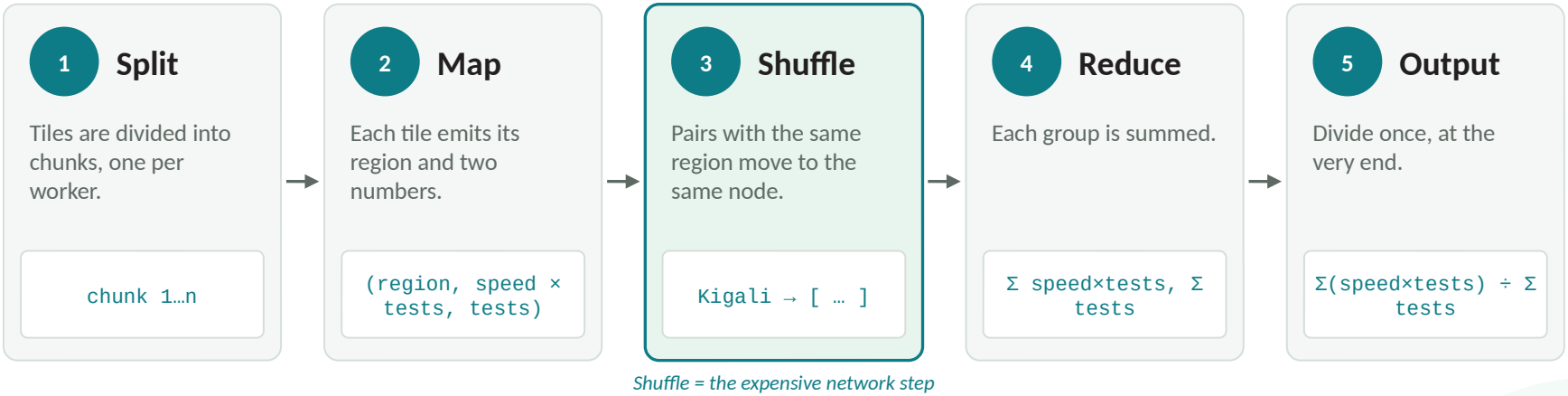
Distributed processing

How do a hundred machines compute one statistic as if they were one?



MapReduce: the idea inside every engine

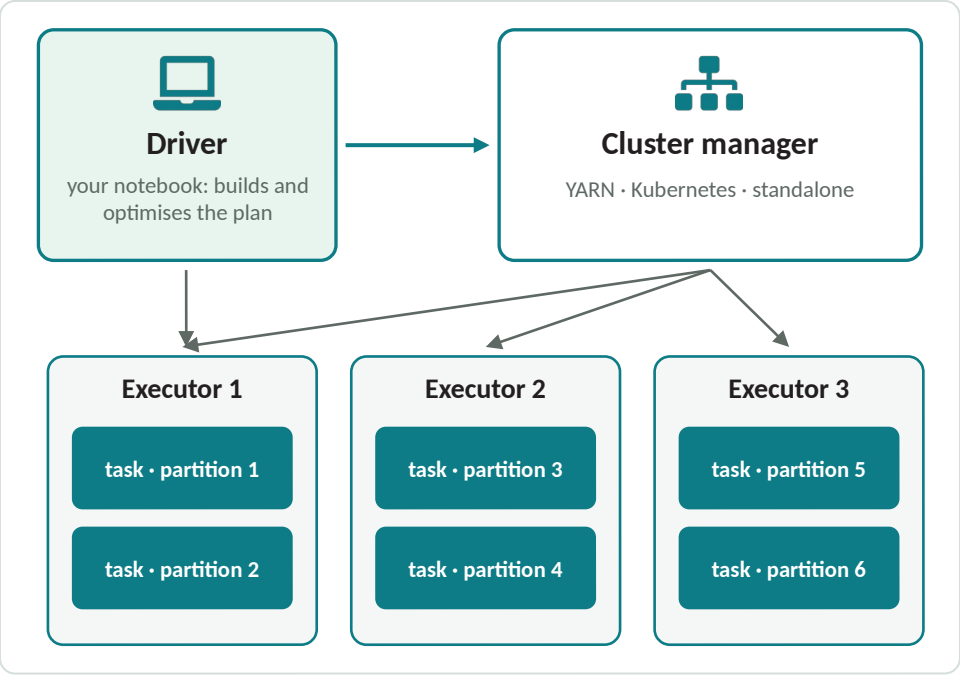
Worked example — tests-weighted mean download speed per region, from Ookla tiles



Shuffle = the expensive network step

Never average the averages. Each Ookla tile is already a mean over a different number of tests. Carry sums and counts through the pipeline and divide at the end — it is both statistically correct and perfectly parallel. Medians and quantiles cannot be split this way, so engines approximate them.

Apache Spark: how a job actually runs



Partitions

Data is split into chunks; one task processes one partition.

Lazy evaluation

Transformations are only recorded; an action (write, count) triggers execution.

Optimiser (Catalyst)

Rewrites the whole plan: pushes filters down, picks join strategies.

Shuffle

Group-bys and joins move data across the network — the usual bottleneck.

Sources: Apache Spark documentation (cluster mode overview); Zaharia et al., Communications of the ACM, 2016.

Spark in practice: a connectivity indicator

PYSPARK

```
from pyspark.sql import SparkSession, functions as F
spark = SparkSession.builder.appName("connectivity").getOrCreate()

# 1. Read validated tiles (silver): already joined to admin units
tiles = spark.read.parquet("s3://nso-lake/silver/ookla_admin/")

# 2. Tests-weighted mean speed + coverage diagnostics
kpi = (tiles
  .filter(F.col("tests") > 0)
  .groupBy("country", "admin1", "year", "quarter")
  .agg((F.sum(F.col("avg_d_kbps") * F.col("tests"))
        / F.sum("tests")).alias("dl_kbps_weighted"),
       F.sum("tests").alias("n_tests"),
       F.countDistinct("quadkey").alias("n_tiles")))

# 3. Publish to the gold zone, one folder per quarter
kpi.write.mode("overwrite") \
   .partitionBy("year", "quarter") \
   .parquet("s3://nso-lake/gold/connectivity/")
```

Identical at any scale

Same script on a laptop sample (local mode) or a full continental history. Only the cluster size changes.

Statistics first

n_tests and n_tiles travel with the estimate, so thin regional figures can be flagged before publication.

Familiar grammar

Filter, group, aggregate, write — the same verbs as SQL, R dplyr or pandas.

Column names follow the Ookla Open Data schema; paths and table names are illustrative.

Batch or streaming: choosing the time model

Batch

Bounded data, processed on a schedule

- Latency: hours to days
- Simple to reason about and to re-run
- Natural fit for monthly, quarterly and annual statistics
- Tools: Spark, DuckDB, dbt, SQL warehouses

Streaming

Unbounded data, processed continuously

- Latency: seconds
- Needs windows, watermarks and event-time logic
- Fit for nowcasting, monitoring, web price collection
- Tools: Apache Kafka, Apache Flink, Spark Structured Streaming

Tumbling windows (5 min)



Each event belongs to exactly one window; each window yields one aggregate.

The one question to ask: does any decision change if the number arrives one hour later? If not, choose batch — it is cheaper, simpler and easier to audit. Quarterly Ookla indicators and monthly NTL composites are batch problems.

Right-size the engine: the single-node renaissance

Laptop in-memory
Up to a few GB · pandas · R data.frame

Single node, columnar
Up to hundreds of GB · DuckDB · Polars · Apache Arrow

Cluster or cloud warehouse
Terabytes, many users · Spark · Trino · BigQuery

Streaming platform
Continuous flows · Kafka + Flink

DUCKDB SQL

```
-- DuckDB: query Ookla in place, no cluster
INSTALL httpfs; LOAD httpfs;
SET s3_region = 'us-west-2';
SELECT count(*) AS tiles,
       sum(tests) AS tests
FROM read_parquet('s3://ookla-open-data/
  parquet/performance/type=mobile/
  year=2025/quarter=4/*.parquet');
```

Only the columns the query needs are read over the network.

The most expensive mistake: running and maintaining a cluster for data that fits on one machine. Measure first, then scale out.

Thresholds are rules of thumb. Sources: DuckDB documentation (httpfs, read_parquet); Ookla bucket region per AWS Open Data Registry.

SECTION 04 OF 06

04

Indexing & search

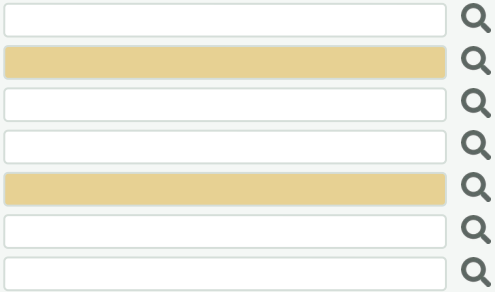
Why can a search engine answer in milliseconds what a full scan needs minutes to find?



From scanning to indexing

Full scan

Read every record, test each one



Work grows with data size: 10× the tiles, ~10× the time.

Inverted index

Look up the value, jump to the records

`admin1 = "Kigali"`	→ doc 2, 5, 9, 14…
`admin1 = "Huye"`	→ doc 1, 7, 11…
`quarter = 2025-Q4`	→ doc 5, 7, 9…

B-tree

Relational databases — ranges and sorting

Inverted index

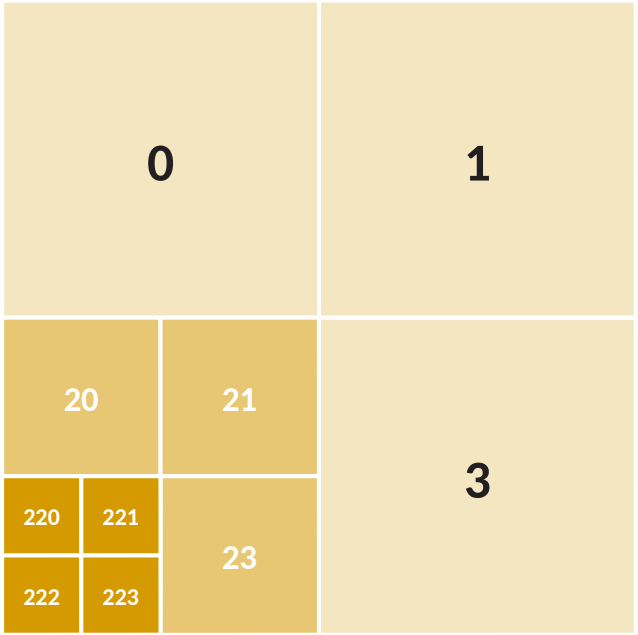
Search engines — terms, keywords, filters

Spatial index

Quadkey, geohash, BKD tree — "what is near / inside"

An index trades write-time work and disk space for read-time speed. Document IDs are illustrative.

The quadkey: a spatial index you already hold



Each level adds one digit (0–3)

16 digits

A zoom-16 quadkey ≈ one 611 m Ookla tile

Prefix = neighbourhood

Keep the first 12 digits to aggregate into ≈ 9.8 km cells — no GIS function needed

Stable identifier

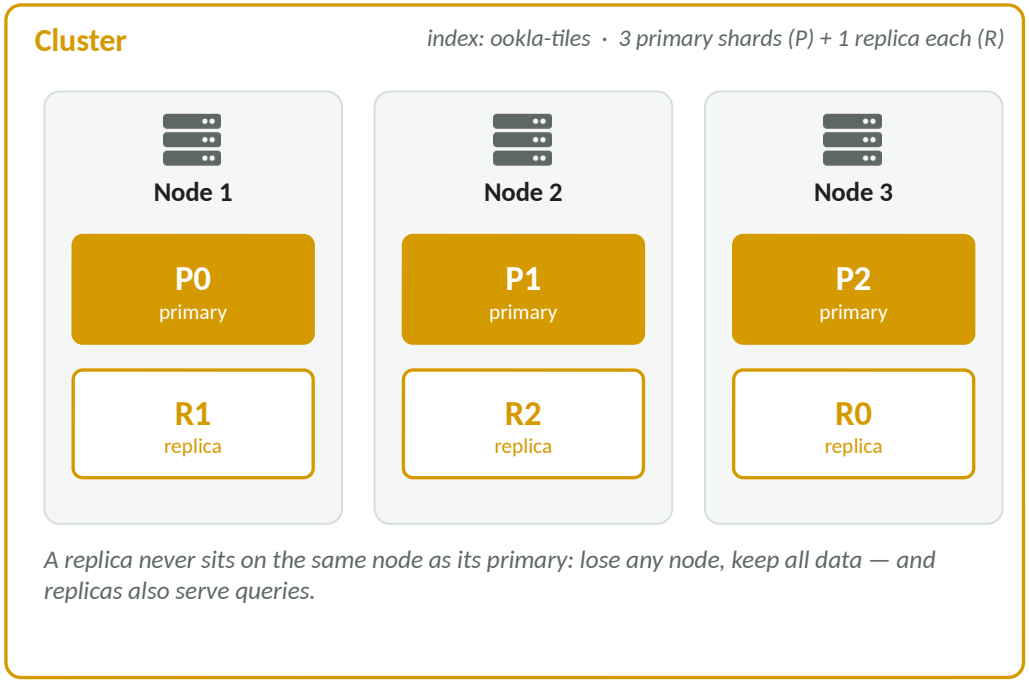
Join the same tile across quarters (2019 → today) on quadkey alone

True shapes → geo_shape

Tile polygon vs district boundary: Elasticsearch indexes shapes in a BKD tree

Sources: Ookla Open Data README; Bing Maps tile system; Elasticsearch geo_shape documentation. Tile width = 40,075 km ÷ 2^zoom.

Elasticsearch anatomy in one picture



If you think in SQL…

Table	→	**Index**
Row	→	**Document (JSON)**
Schema	→	**Mapping**
Column type	→	**Field type: keyword, integer, geo_shape…**
WHERE	→	**Query (bool, term, range, geo_shape)**
GROUP BY	→	**Aggregation (terms, weighted_avg…)**

Source: Elasticsearch Reference (clusters, indices, shards and mappings).

Mapping and query: what you will type in the lab

1 · MAPPING — THE SCHEMA

```
PUT ookla-tiles
{ "mappings": { "properties": {
  "quadkey":    { "type": "keyword" },
  "admin1":     { "type": "keyword" },
  "quarter":    { "type": "date" },
  "avg_d_kbps": { "type": "integer" },
  "tests":      { "type": "integer" },
  "devices":    { "type": "integer" },
  "tile":       { "type": "geo_shape" }
}}}

// keyword  = exact value: filter, group
// geo_shape = the real tile polygon
```

2 · QUERY + AGGREGATION — BY REGION

```
GET ookla-tiles/_search
{ "size": 0,
  "query": { "geo_shape": { "tile": {
    "shape": { "type": "envelope",
      "coordinates": [[28.8, -1.0], [30.9, -2.9]] },
    "relation": "intersects" }}},
  "aggs": { "by_region": {
    "terms": { "field": "admin1" },
    "aggs": { "dl_weighted": { "weighted_avg": {
      "value":  { "field": "avg_d_kbps" },
      "weight": { "field": "tests" } }}}}}}

// size 0 = return only the aggregation
// envelope = [top-left], [bottom-right], lon first
```

Sources: Elasticsearch Reference (geo_shape query, weighted_avg aggregation). Bounding box approximates Rwanda; the lab cluster mapping may differ.

When is a search engine worth the extra complexity?

	pandas	DuckDB on Parquet	Elasticsearch
Best for	One analyst, exploratory work	Heavy analytical SQL on one machine	Many users, interactive filters and maps
Memory	Whole dataset in RAM	Streams columns; can spill to disk	Distributed across nodes
Geo & text	Via GeoPandas, in memory	Spatial extension	Native geo_shape and full-text
Interactive latency	Fine while data is small	Fast for scans and joins	Designed for sub-second queries
Operating cost	None	Very low	A cluster to run, secure and monitor

A search engine is an index, not a system of record. Keep the lakehouse as the source of truth and rebuild the index from it.

Qualitative comparison — Lab part 2 asks you to measure query time and memory footprint yourself.

SECTION 05 OF 06

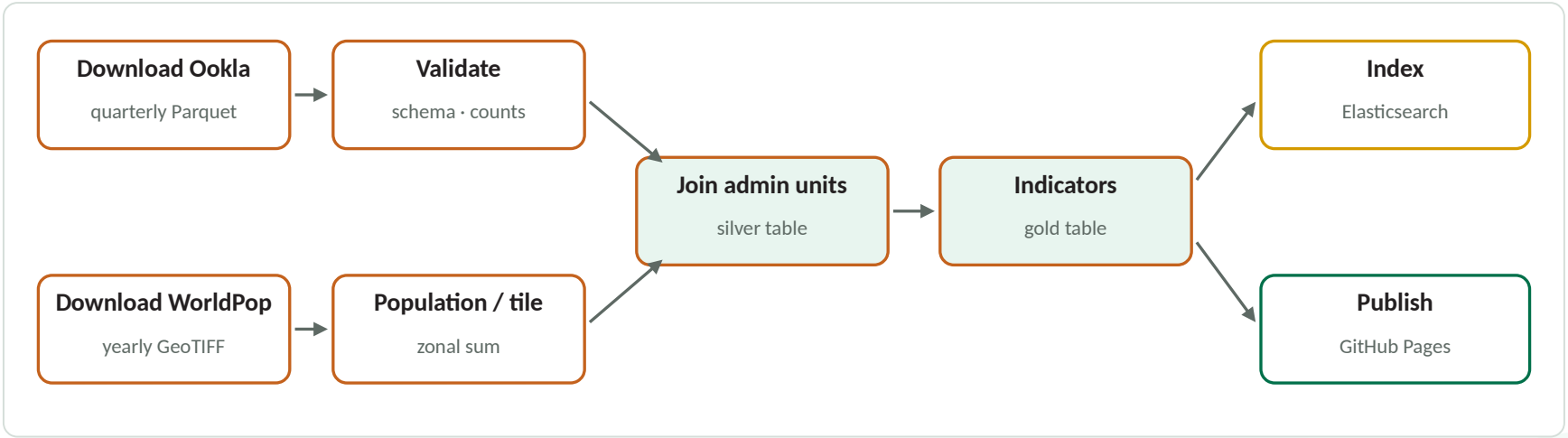
05

Orchestration

Who runs the pipeline at 2 a.m. — and what happens when a step fails?



A pipeline is a graph of tasks



Schedule · Dependencies · Retries · Alerts · Backfill

Tools, simplest first: cron · GitHub Actions → Apache Airflow · Dagster · Prefect · dbt for SQL transformations

Pipelines as code, with quality gates

APACHE AIRFLOW (TASKFLOW API)

```
from airflow.decorators import dag
import pendulum
# download, validate, … are @task functions

@dag(schedule="@quarterly",
     start_date=pendulum.datetime(2019, 1, 1),
     catchup=True,            # = backfill
     default_args={"retries": 2})
def ookla_connectivity():
    raw    = download_tiles()
    ok     = validate(raw)       # quality gate
    pop    = population_per_tile()
    silver = join_admin(ok, pop)
    gold   = indicators(silver)
    index_elasticsearch(gold)
    publish_github(gold)

ookla_connectivity()
```

Quality gates, written as tests

- ✓ `tests > 0 for every tile`
- ✓ `quadkey unique within each quarter`
- ✓ `avg_d_kbps within a plausible range`
- ✓ `every tile assigned to exactly one admin unit`
- ✓ `tile count within ±X % of last quarter`

On failure: stop, alert a person, never publish quietly. Tools: Great Expectations · dbt tests · Soda · pandera

Source: Apache Airflow documentation (TaskFlow API, catchup). Thresholds are set by the methodologist.

SECTION 06 OF 06

06

Metadata & versioning

Can you prove, months later, exactly how a published number was produced?



Three layers of metadata, one chain of lineage

Technical

Schema, types, formats, partitions, size, location

Parquet schema · Iceberg catalogue

Statistical

Concepts, definitions, classifications, methods, limitations

SDMX · DDI · GSIM

Operational

Lineage, run logs, quality results, access history

Pipeline logs · catalogue lineage

Lineage: every published number traces back to exact inputs



Catalogue tools: DataHub, OpenMetadata, CKAN; dataset descriptions: W3C DCAT. Identifiers shown are illustrative.

Version three things, not one



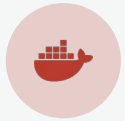
Code

- Git repository, reviewed pull requests
- Tagged releases (v1.2.0)
- Automated tests in CI



Data

- Table snapshots: Iceberg, Delta Lake
- File versioning: DVC, lakeFS
- Bronze zone is never overwritten



Environment

- Pinned dependencies: lock files
- Conda environments or containers
- Same inputs + same code + same env = same number



The release = what Days 3–5 will produce

Tagged GitHub release + README (method, sources, licence, limitations) + citable DOI via Zenodo + named maintainer

Sources: GitHub–Zenodo integration documentation; DVC and lakeFS documentation.

A platform an NSO can grow into

START	One server or laptop
Storage	Folders or MinIO + Parquet
Processing	DuckDB · Polars · pandas
Search	Optional
Orchestration	cron · GitHub Actions
Metadata	README + provenance files
Versioning	Git + GitHub

grow on evidence →

SCALE	Shared national or regional platform
Storage	Object storage + Iceberg / Delta
Processing	Spark · Trino
Search	Elasticsearch cluster
Orchestration	Airflow · Dagster
Metadata	DataHub / OpenMetadata, SDMX
Versioning	Snapshots, containers, CI/CD

Pool effort: regional big data hubs and shared national platforms let countries share platforms, code and skills.

DECISION GUIDE

Choosing the right engine

If your situation is...	Reach for	Avoid
Data fits in memory, one analyst	**pandas, R, DuckDB**	Standing up a cluster
Tens to hundreds of GB, complex SQL	**DuckDB or Polars on one strong server**	Copying CSVs between laptops
Terabytes, several teams, shared tables	**Lakehouse + Spark or Trino**	Ungoverned network file shares
Many users filtering by place and time	**Elasticsearch fed from the lakehouse**	Using the index as the only copy
Sub-minute latency genuinely needed	**Kafka + Flink**	Streaming for quarterly statistics
Confidential microdata (CDRs, tax)	**On-premise or sovereign enclave**	Public cloud without a legal basis

Choose for the data you have, not the data you imagine having in five years.

KEY TAKEAWAYS

Six things to remember

01

Ingest honestly

Keep raw data untouched; record source, time and licence from the first byte.

02

Store in zones and columns

Parquet + bronze / silver / gold gives speed and a clear production chain.

03

Compute with sums, not means

Parallel engines need decomposable statistics; right-size before you scale out.

04

Index for interaction

A search engine answers fast — but it is an index, never the source of truth.

05

Automate with gates

Pipelines run on schedule and stop on failed checks — no quiet publishing.

06

Make every number traceable

Version code, data and environment; document methods and limitations.

QUESTIONS & DISCUSSION

Thank you



Sources and further reading

Data used in the labs

- Ookla — Speedtest Global Performance Open Data, github.com/teamookla/ookla-open-data
- WorldPop — Open spatial demographic data, worldpop.org
- NASA — Black Marble night-time lights (VNP46 products)

Foundations

- Dean & Ghemawat, "MapReduce", OSDI 2004
- Zaharia et al., "Apache Spark: A Unified Engine for Big Data Processing", CACM 2016
- Armbrust et al., "Lakehouse", CIDR 2021
- McKinney, "Apache Arrow and the 10 Things I Hate About pandas", 2017

Technical documentation

- Apache Hadoop HDFS Architecture · Apache Parquet
- Apache Spark · Apache Iceberg · Delta Lake
- DuckDB · Elasticsearch Reference
- Apache Airflow · GitHub–Zenodo integration

Official statistics frameworks

- UNECE — Generic Statistical Business Process Model (GSBPM)
- UNECE — Quality Framework for Statistics Derived from Big Data
- SDMX · DDI · GSIM · W3C DCAT
- UN Committee of Experts on Big Data and Data Science for Official Statistics